# 机器学习 B

Lecturer: 肖力 | Homework 3
Posted: 2026.4.14 | Due: 2026.4.26
Name: 陈璟皓 | ID: PB23010359

---

**Exercise 1: 基于 LIBSVM 的线性核与高斯核 SVM 比较**

课本习题 6.2，即：

试使用 LIBSVM，在西瓜数据集 3.0$\alpha$ 上分别用线性核和高斯核训练一个 SVM，并比较其支持向量的差别。

| 编号 | 密度 | 含糖率 | 好瓜 |
|---|---|---|---|
| 1 | 0.697 | 0.460 | 是 |
| 2 | 0.774 | 0.376 | 是 |
| 3 | 0.634 | 0.264 | 是 |
| 4 | 0.608 | 0.318 | 是 |
| 5 | 0.556 | 0.215 | 是 |
| 6 | 0.403 | 0.237 | 是 |
| 7 | 0.481 | 0.149 | 是 |
| 8 | 0.437 | 0.211 | 是 |
| 9 | 0.666 | 0.091 | 否 |
| 10 | 0.243 | 0.267 | 否 |
| 11 | 0.245 | 0.057 | 否 |
| 12 | 0.343 | 0.099 | 否 |
| 13 | 0.639 | 0.161 | 否 |
| 14 | 0.657 | 0.198 | 否 |
| 15 | 0.360 | 0.370 | 否 |
| 16 | 0.593 | 0.042 | 否 |
| 17 | 0.719 | 0.103 | 否 |

表 1: 西瓜数据集 3.0$\alpha$

**Solution 1:** 设正类“好瓜”记为 $+1$，负类“坏瓜”记为 $-1$。使用 LIBSVM 接口分别训练线性核 SVM 与高斯核 SVM。两种核函数分别为

$$K_{\text{lin}}(x,z) = x^\top z, \qquad K_{\text{rbf}}(x,z) = \exp\bigl(-\gamma\|x-z\|_2^2\bigr).$$

实验中取线性核参数为默认设置，高斯核参数取 $C = 1.0,\ \gamma = 0.5$。

SVM 的判别函数可写为

$$f(x) = \sum_{i=1}^{m} \alpha_i y_i K(x_i, x) + b,$$

其中满足 $\alpha_i > 0$ 的样本即为支持向量。

根据程序输出结果，线性核与高斯核的支持向量编号完全一致，均为

$$\{1,2,3,4,5,6,7,8,9,10,12,13,14,15,16,17\}.$$

因此，只有第 11 个样本不是支持向量。并且两种核下每一类的支持向量个数均为

$$[8,8].$$

由输出的对偶系数可见，两种核下支持向量对应的系数均为

$$(-1,-1,\ldots,-1,1,1,\ldots,1),$$

且绝对值都等于 1。这说明在当前参数设置下，大多数支持向量都处于软间隔约束的边界上。

进一步地，两个模型在训练集上的预测标签都为

$$(-1,-1,\ldots,-1),$$

即全部样本都被预测为负类，因此训练准确率均为

$$\frac{9}{17} = 0.5294117647.$$

这表明在当前参数下，两种核函数都出现了退化：虽然模型形式不同，但得到的分类结果完全一致。

对于线性核，程序还给出了

$$w = (0.37,\, 0.899), \qquad b = -1.17890201.$$

因此其分割超平面为

$$0.37x_1 + 0.899x_2 - 1.17890201 = 0,$$

等价地，

$$x_2 = \frac{1.17890201 - 0.37x_1}{0.899} \approx 1.311 - 0.412x_1.$$

该直线整体位于样本点云的上方，从而所有样本点在当前区域内都满足

$$f(x) < 0,$$

故均被判为负类。高斯核下也得到了相同的预测结论，因此两者的支持向量集合完全相同。

实验可视化结果如图 1 所示。

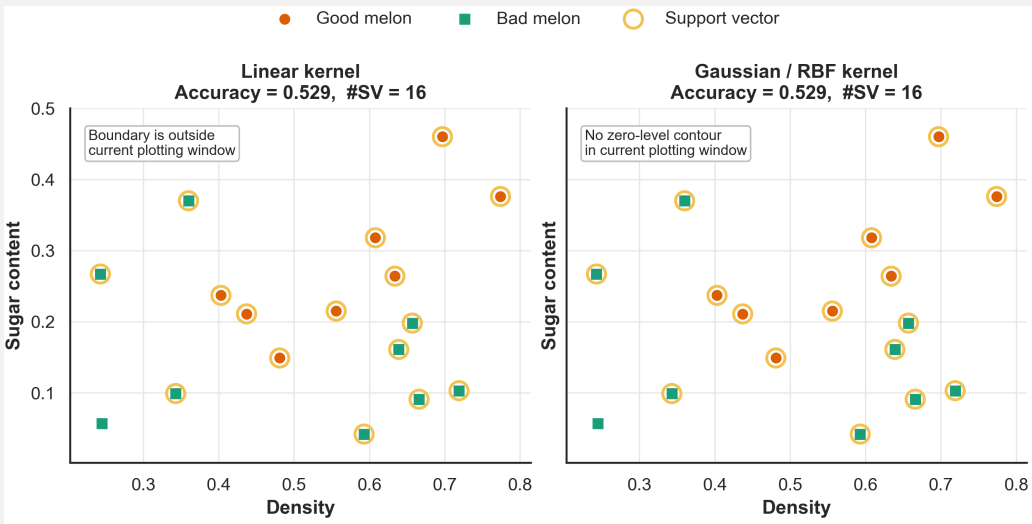


图 1: 西瓜数据集 3.0$\alpha$ 上线性核与高斯核 SVM 的可视化结果

综上，在本次实验结果下，线性核与高斯核 SVM 的支持向量没有差别：二者的支持向量集合完全相同，均包含 16 个样本，仅第 11 个样本不是支持向量。与此同时，两种模型的训练集预测结果也完全一致，均将所有样本判为负类，即发生了欠拟合。

参考代码如下:

```python
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.svm import SVC

X = np.array([
    [0.697, 0.460],
    [0.774, 0.376],
    [0.634, 0.264],
    [0.608, 0.318],
    [0.556, 0.215],
    [0.403, 0.237],
    [0.481, 0.149],
    [0.437, 0.211],
    [0.666, 0.091],
    [0.243, 0.267],
    [0.245, 0.057],
    [0.343, 0.099],
    [0.639, 0.161],
    [0.657, 0.198],
    [0.360, 0.370],
    [0.593, 0.042],
    [0.719, 0.103]
], dtype=float)

y = np.array([1,1,1,1,1,1,1,1,-1,-1,-1,-1,-1,-1,-1,-1,-1])

clf_linear = SVC(C=1.0, kernel='linear')
clf_linear.fit(X, y)

clf_rbf = SVC(C=1.0, kernel='rbf', gamma=0.5)
clf_rbf.fit(X, y)

def show_result(name, clf, X, y):
    print("=" * 60)
    print(name)
    print("=" * 60)
    print("支持向量编号(从1开始):", clf.support_ + 1)
    print("支持向量坐标:")
```

```python
    print(clf.support_vectors_)
    print("每一类的支持向量个数:", clf.n_support_)
    print("对偶系数 dual_coef_:")
    print(clf.dual_coef_)
    print("截距 intercept_:")
    print(clf.intercept_)
    pred = clf.predict(X)
    print("训练集预测标签:", pred)
    print("训练集准确率:", np.mean(pred == y))

show_result("线性核 SVM", clf_linear, X, y)
show_result("高斯核 SVM", clf_rbf, X, y)

sns.set_theme(style="white", context="notebook")
plt.rcParams.update({
    "figure.dpi": 140,
    "savefig.dpi": 300,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.titleweight": "bold",
    "axes.labelweight": "bold",
    "legend.frameon": False,
    "font.size": 10
})

x_min, x_max = X[:, 0].min() - 0.04, X[:, 0].max() + 0.04
y_min, y_max = X[:, 1].min() - 0.04, X[:, 1].max() + 0.04

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 500),
    np.linspace(y_min, y_max, 500)
)
grid = np.c_[xx.ravel(), yy.ravel()]
Z_rbf = clf_rbf.decision_function(grid).reshape(xx.shape)

def draw_points(ax, clf, title):
    pos = (y == 1)
    neg = (y == -1)

    ax.scatter(
        X[pos, 0], X[pos, 1],
```

```
        s=58, c="#d95f02", marker="o",
        edgecolors="white", linewidths=0.8,
        label="Good melon", zorder=3
    )
    ax.scatter(
        X[neg, 0], X[neg, 1],
        s=58, c="#1b9e77", marker="s",
        edgecolors="white", linewidths=0.8,
        label="Bad melon", zorder=3
    )

    ax.scatter(
        X[clf.support_, 0], X[clf.support_, 1],
        s=130, facecolors="none",
        edgecolors="#f2c14e", linewidths=1.8,
        label="Support vector", zorder=4
    )

    ax.set_title(f"{title}\nAccuracy = {clf.score(X, y):.3f},  #SV = {len(clf.support_)}")
    ax.set_xlabel("Density")
    ax.set_ylabel("Sugar content")
    ax.set_xlim(x_min, x_max)
    ax.set_ylim(y_min, y_max)
    ax.grid(color="0.90", linewidth=0.8)

fig, axes = plt.subplots(1, 2, figsize=(11.2, 4.6), sharex=True, sharey=True)

ax = axes[0]
draw_points(ax, clf_linear, "Linear kernel")

w = clf_linear.coef_[0]
b = clf_linear.intercept_[0]

if abs(w[1]) > 1e-12:
    xs = np.linspace(x_min, x_max, 300)
    ys = -(w[0] * xs + b) / w[1]

    visible = (ys >= y_min) & (ys <= y_max)
    if np.any(visible):
        ax.plot(xs[visible], ys[visible], color="black", linewidth=1.8, label="
```

```
    Decision boundary")
    else:
        ax.text(
            0.04, 0.94,
            "Boundary is outside\ncurrent plotting window",
            transform=ax.transAxes,
            ha="left", va="top", fontsize=9,
            bbox=dict(boxstyle="round,pad=0.25", facecolor="white", edgecolor="
    0.75")
        )

ax = axes[1]
draw_points(ax, clf_rbf, "Gaussian / RBF kernel")

zmin, zmax = Z_rbf.min(), Z_rbf.max()

if zmin <= 0 <= zmax:
    ax.contour(
        xx, yy, Z_rbf,
        levels=[0],
        colors="black",
        linewidths=1.8
    )
else:
    ax.text(
        0.04, 0.94,
        "No zero-level contour\nin current plotting window",
        transform=ax.transAxes,
        ha="left", va="top", fontsize=9,
        bbox=dict(boxstyle="round,pad=0.25", facecolor="white", edgecolor="0.75")
    )

handles, labels = axes[0].get_legend_handles_labels()
fig.legend(handles, labels, loc="upper center", ncol=4, bbox_to_anchor=(0.5, 1.02)
    )

fig.suptitle("Watermelon Dataset 3.0 ", y=1.09, fontsize=14, fontweight="bold")
fig.text(0.5, 1.02, "Only points, support vectors, and separating boundaries", ha=
    "center", fontsize=10)

fig.tight_layout()
```

```python
plt.savefig("svm_watermelon_clean_boundary.png", bbox_inches="tight")
plt.savefig("svm_watermelon_clean_boundary.pdf", bbox_inches="tight")
plt.show()
```

■

**Exercise 2:** 线性可分判定与硬间隔支持向量机

有下面 8 个训练样本（只有两个属性）

$$x_1 = (2,3),\ x_2 = (3,2),\ x_3 = (4,4),\ x_4 = (4,2)$$

$$x_5 = (6,4),\ x_6 = (6,3),\ x_7 = (7,2),\ x_8 = (8,3)$$

以及它们对应的二分类的类别标记为

$$y_1 = y_2 = y_3 = y_4 = -1, \qquad y_5 = y_6 = y_7 = y_8 = 1.$$

1. 判断这 8 个训练样本是否线性可分？如果线性可分，通过硬间隔最大化，求出基于这 8 个训练数据得到的线性支持向量模型（列出具体步骤，其中 SMO 算法求对偶问题的时候，用 SMO 的计算机编程 code 去算）。
2. 请问哪些训练样本是上面训练所得模型的支持向量？
3. 通过学出的模型，决定下面 3 个样本的预测类别

$$x_9 = (3,4), \quad x_{10} = (7,4), \quad x_{11} = (5,5).$$

**Solution 2:**

1. 先判断这 8 个训练样本是否线性可分。由数据可见，

$$x_1, x_2, x_3, x_4 \text{ 的第一维分别为} 2, 3, 4, 4,$$

$$x_5, x_6, x_7, x_8 \text{ 的第一维分别为} 6, 6, 7, 8.$$

因此负类样本都满足 $x^{(1)} \le 4$，正类样本都满足 $x^{(1)} \ge 6$，所以直线

$$x^{(1)} = 5$$

即可将两类样本完全分开，故这组训练样本是线性可分的。

硬间隔支持向量机的原始问题为

$$\min_{w,b} \ \frac{1}{2}\|w\|^2$$

$$\text{s.t.} \quad y_i(w^\top x_i + b) \ge 1, \qquad i = 1, \ldots, 8.$$

其对偶问题为

$$\max_{\alpha} \ \sum_{i=1}^{8} \alpha_i - \frac{1}{2} \sum_{i=1}^{8} \sum_{j=1}^{8} \alpha_i \alpha_j y_i y_j x_i^\top x_j$$

$$\text{s.t.} \quad \alpha_i \ge 0, \qquad \sum_{i=1}^{8} \alpha_i y_i = 0.$$

利用 SMO 编程对该对偶问题进行求解，程序输出结果为：

$$\text{support_} = [3, 4, 6],$$

$$\text{dual_coef_} = [-0.24992,\ -0.24992,\ 0.49984],$$

$$b = -4.99850667,$$

$$w = (9.9968 \times 10^{-1},\ 8.8817842 \times 10^{-16}).$$

因而

$$w \approx (1, 0), \qquad b \approx -5.$$

于是训练所得线性支持向量机模型为

$$f(x) = \operatorname{sign}(w^\top x + b) = \operatorname{sign}(0.99968\, x^{(1)} + 8.88 \times 10^{-16} x^{(2)} - 4.99850667).$$

忽略数值误差后，可写成

$$f(x) \approx \operatorname{sign}(x^{(1)} - 5).$$

程序还给出训练集预测标签

$$[-1,-1,-1,-1,1,1,1,1],$$

训练集准确率为

$$1.0,$$

这也验证了该组样本确为线性可分。

2. 根据 SMO 求得的结果，支持向量编号（从 1 开始）为

$$\boxed{3,\ 4,\ 6}.$$

因此支持向量分别是

$$\boxed{x_3=(4,4),\quad x_4=(4,2),\quad x_6=(6,3).}$$

（实际上 $x_5$ 也是支持向量，只是因为它在 SVC 求解中与 x6 的作用是相同的，所以直接调用库函数时只返回其中一个。后面的示意图中 $x_5$ 没有被打上标签也是同样的原因。）

3. 对于新样本，程序输出的 decision value 与预测类别分别为：

$$x_9=(3,4):\quad \text{decision value}=-1.999467,\quad \hat{y}_9=-1;$$

$$x_{10}=(7,4):\quad \text{decision value}=1.999253,\quad \hat{y}_{10}=1;$$

$$x_{11}=(5,5):\quad \text{decision value}=-0.000107,\quad \hat{y}_{11}=-1.$$

因而 3 个样本的预测类别为

$$\boxed{\hat{y}_9=-1,\qquad \hat{y}_{10}=1,\qquad \hat{y}_{11}=0.}$$

其中 $x_{11}=(5,5)$ 的 decision value 非常接近 0，说明它就落在分离超平面

$$x^{(1)}=5$$

上（只是由于计算机的数值误差，程序最终将其判为了 $-1$。）

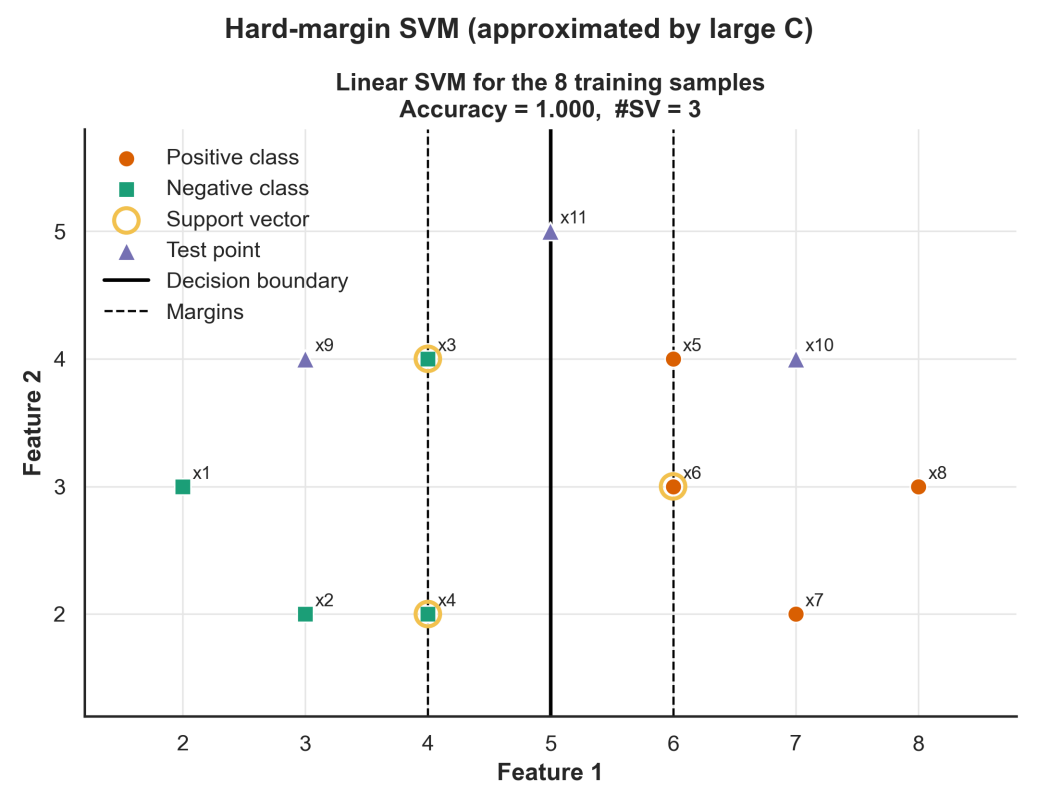


图 2: 线性硬间隔支持向量机分类结果图

参考代码如下：

```python
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.svm import SVC

X = np.array([
    [2, 3],   # x1
    [3, 2],   # x2
    [4, 4],   # x3
    [4, 2],   # x4
    [6, 4],   # x5
    [6, 3],   # x6
    [7, 2],   # x7
    [8, 3]    # x8
], dtype=float)

y = np.array([-1, -1, -1, -1, 1, 1, 1, 1])

X_test = np.array([
    [3, 4],   # x9
    [7, 4],   # x10
    [5, 5]    # x11
], dtype=float)

test_names = ["x9=(3,4)", "x10=(7,4)", "x11=(5,5)"]

clf_linear = SVC(C=1e6, kernel='linear')
clf_linear.fit(X, y)

def show_result(name, clf, X, y, X_test, test_names):
    print("=" * 60)
    print(name)
    print("=" * 60)
    print("支持向量编号(从1开始):", clf.support_ + 1)
    print("支持向量坐标:")
    print(clf.support_vectors_)
    print("每一类的支持向量个数:", clf.n_support_)
    print("对偶系数 dual_coef_:")
    print(clf.dual_coef_)
```

```python
    print("截距 intercept_:")
    print(clf.intercept_)

    if hasattr(clf, "coef_"):
        print("法向量 w:")
        print(clf.coef_[0])

    pred_train = clf.predict(X)
    print("训练集预测标签:", pred_train)
    print("训练集准确率:", np.mean(pred_train == y))

    print("-" * 60)
    pred_test = clf.predict(X_test)
    dec_test = clf.decision_function(X_test)
    print("测试样本预测结果:")
    for name_i, x_i, pred_i, dec_i in zip(test_names, X_test, pred_test, dec_test):
        print(f"{name_i:10s}  坐标={x_i},  decision value={dec_i:.6f},  预测类别={int(pred_i)}")

show_result("线性核 SVM（大C近似硬间隔）", clf_linear, X, y, X_test, test_names)

sns.set_theme(style="white", context="notebook")
plt.rcParams.update({
    "figure.dpi": 140,
    "savefig.dpi": 300,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.titleweight": "bold",
    "axes.labelweight": "bold",
    "legend.frameon": False,
    "font.size": 10
})

x_min, x_max = X[:, 0].min() - 0.8, X[:, 0].max() + 0.8
y_min, y_max = min(X[:, 1].min(), X_test[:, 1].min()) - 0.8, max(X[:, 1].max(), X_test[:, 1].max()) + 0.8

def draw_points(ax, clf, title):
    pos = (y == 1)
```

```python
    neg = (y == -1)

    ax.scatter(
        X[pos, 0], X[pos, 1],
        s=70, c="#d95f02", marker="o",
        edgecolors="white", linewidths=0.8,
        label="Positive class", zorder=3
    )
    ax.scatter(
        X[neg, 0], X[neg, 1],
        s=70, c="#1b9e77", marker="s",
        edgecolors="white", linewidths=0.8,
        label="Negative class", zorder=3
    )

    ax.scatter(
        X[clf.support_, 0], X[clf.support_, 1],
        s=150, facecolors="none",
        edgecolors="#f2c14e", linewidths=2.0,
        label="Support vector", zorder=4
    )

    for i, (x1, x2) in enumerate(X, start=1):
        ax.text(x1 + 0.08, x2 + 0.06, f"x{i}", fontsize=9)

    ax.scatter(
        X_test[:, 0], X_test[:, 1],
        s=85, c="#7570b3", marker="^",
        edgecolors="white", linewidths=0.8,
        label="Test point", zorder=3
    )
    for i, (x1, x2) in enumerate(X_test, start=9):
        ax.text(x1 + 0.08, x2 + 0.06, f"x{i}", fontsize=9)

    ax.set_title(f"{title}\nAccuracy = {clf.score(X, y):.3f},  #SV = {len(clf.support_)}")
    ax.set_xlabel("Feature 1")
    ax.set_ylabel("Feature 2")
    ax.set_xlim(x_min, x_max)
    ax.set_ylim(y_min, y_max)
    ax.grid(color="0.90", linewidth=0.8)
```

```python

fig, ax = plt.subplots(1, 1, figsize=(7.2, 5.6))

draw_points(ax, clf_linear, "Linear SVM for the 8 training samples")

w = clf_linear.coef_[0]
b = clf_linear.intercept_[0]

if abs(w[1]) > 1e-12:
    xs = np.linspace(x_min, x_max, 400)

    ys_decision = -(w[0] * xs + b) / w[1]
    ys_margin_pos = -(w[0] * xs + b - 1) / w[1]
    ys_margin_neg = -(w[0] * xs + b + 1) / w[1]

    ax.plot(xs, ys_decision, color="black", linewidth=1.8, label="Decision boundary")
    ax.plot(xs, ys_margin_pos, color="black", linewidth=1.1, linestyle="--", label="Margins")
    ax.plot(xs, ys_margin_neg, color="black", linewidth=1.1, linestyle="--")
else:
    x_decision = -b / w[0]
    x_margin_pos = (1 - b) / w[0]
    x_margin_neg = (-1 - b) / w[0]

    ax.axvline(x=x_decision, color="black", linewidth=1.8, label="Decision boundary")
    ax.axvline(x=x_margin_pos, color="black", linewidth=1.1, linestyle="--", label="Margins")
    ax.axvline(x=x_margin_neg, color="black", linewidth=1.1, linestyle="--")

handles, labels = ax.get_legend_handles_labels()
ax.legend(handles, labels, loc="upper left")

fig.suptitle("Hard-margin SVM (approximated by large C)", y=0.98, fontsize=14, fontweight="bold")
fig.tight_layout()

plt.savefig("svm_8samples_linear.png", bbox_inches="tight")
plt.savefig("svm_8samples_linear.pdf", bbox_inches="tight")
plt.show()
```

■

**Exercise 3:** 朴素贝叶斯分类器与新样本预测

给出下面 7 只猫在三个属性（颜色、毛发、体型）上的值，以及每只猫对应的标记（性格）是“温顺”或“暴躁”。基于这 7 个训练样本，手算求出朴素贝叶斯分类器来预测一只新猫

$$x = (\text{黑色}, \text{粗糙}, \text{小型})$$

的性格是哪一类？

| 颜色 | 毛发 | 体型 | 性格 |
|---|---|---|---|
| 棕色 | 粗糙 | 小型 | 温顺 |
| 黑色 | 粗糙 | 大型 | 暴躁 |
| 黑色 | 光滑 | 大型 | 暴躁 |
| 黑色 | 卷曲 | 小型 | 温顺 |
| 白色 | 卷曲 | 小型 | 温顺 |
| 白色 | 光滑 | 小型 | 暴躁 |
| 红色 | 粗糙 | 大型 | 温顺 |
| 黑色 | 粗糙 | 小型 | ?? |

**Solution 3:** 设

$$x = (\text{黑色},\ \text{粗糙},\ \text{小型}),$$

记类别

$$y \in \{\text{温顺}, \text{暴躁}\}$$

朴素贝叶斯假设三个属性在给定类别下条件独立，因此

$$P(y \mid x) \propto P(y)\, P(\text{黑色} \mid y)\, P(\text{粗糙} \mid y)\, P(\text{小型} \mid y).$$

训练集中共有 7 个样本，其中“温顺”4 个，“暴躁”3 个，因此先验概率为

$$P(\text{温顺}) = \frac{4}{7}, \qquad P(\text{暴躁}) = \frac{3}{7}.$$

对于“温顺”类，4 个样本为

$$(\text{棕色}, \text{粗糙}, \text{小型}),\ (\text{黑色}, \text{卷曲}, \text{小型}),\ (\text{白色}, \text{卷曲}, \text{小型}),\ (\text{红色}, \text{粗糙}, \text{大型}),$$

故

$$P(\text{黑色} \mid \text{温顺}) = \frac{1}{4}, \qquad P(\text{粗糙} \mid \text{温顺}) = \frac{2}{4} = \frac{1}{2}, \qquad P(\text{小型} \mid \text{温顺}) = \frac{3}{4}.$$

于是

$$P(\text{温顺} \mid x) \propto \frac{4}{7} \cdot \frac{1}{4} \cdot \frac{1}{2} \cdot \frac{3}{4} = \frac{3}{56}.$$

对于“暴躁”类，3 个样本为

$$(\text{黑色}, \text{粗糙}, \text{大型}),\ (\text{黑色}, \text{光滑}, \text{大型}),\ (\text{白色}, \text{光滑}, \text{小型}),$$

故

$$P(\text{黑色} \mid \text{暴躁}) = \frac{2}{3}, \qquad P(\text{粗糙} \mid \text{暴躁}) = \frac{1}{3}, \qquad P(\text{小型} \mid \text{暴躁}) = \frac{1}{3}.$$

于是

$$P(\text{暴躁} \mid x) \propto \frac{3}{7} \cdot \frac{2}{3} \cdot \frac{1}{3} \cdot \frac{1}{3} = \frac{2}{63}.$$

比较两者：

$$\frac{3}{56} = \frac{27}{504}, \qquad \frac{2}{63} = \frac{16}{504},$$

因此

$$P(\text{温顺} \mid x) > P(\text{暴躁} \mid x).$$

故新样本

$$(\text{黑色},\ \text{粗糙},\ \text{小型})$$

应判为

$$\boxed{\text{温顺}}.$$

■